



ITV CyL

PROYECTO INTERMODULAR DE DESARROLLO DE APLICACIONES MULTIPLATAFORMA

Autor: David Andrés Pérez y Juan Francisco Herreros González

Tutor(es): Aída Rodríguez y Elena Sarmentero

Curso: 2025-2026

ÍNDICE

1. INTRODUCCIÓN	3
2. FASE DE ANÁLISIS	5
2.1 Descripción de los actores y roles	5
2.2 Especificación de requisitos	5
2.2.1 Requisitos Funcionales	5
2.2.2 Requisitos no funcionales	7
3. FASE DE DISEÑO	10
3.1 Casos de uso.....	10
3.2 Análisis de persistencia	11
3.2.1 Diagrama Entidad-Relación.....	11
3.3 Diseño funcional y lógica del sistema	12
3.3.1 Arquitectura del sistema.....	12
3.3.2 Scripts desarrollados.....	14
3.4 Análisis diseño interfaz	18
3.4.1 Elección de JAVA FX y herramientas de desarrollo	18
3.4.2 Conexión con la base de datos	19
3.4.3 Arquitectura del <i>frontend</i>	19
3.4.4 Prototipo de Baja Fidelidad.....	20
3.4.5 Prototipo de Media Fidelidad	20
4. FASE DE PRUEBAS.....	21
5. DESPLIEGUE DE LA APLICACIÓN.....	22
6. DOCUMENTACIÓN DE LA APLICACIÓN	23
7. CONCLUSIONES	24
8. BIBLIOGRAFÍA.....	25
ANEXOS.....	27
Anexo I – Desarrollo de las tareas realizadas	27
Anexo II – Uso de la Inteligencia Artificial en el desarrollo del proyecto	27
1. Herramientas de inteligencia artificial utilizadas	27
2. Momentos del proyecto en los que se han utilizado.....	27
3. Reporte de Promts.....	28
4. Aportación de la IA al proyecto.....	33

1. INTRODUCCIÓN

La Inspección Técnica de Vehículos, conocida popularmente como ITV, es un proceso de revisión periódica y obligatoria en España al que deben someterse todos los vehículos en circulación. Su objetivo es verificar que los vehículos cumplen con los requisitos técnicos y medioambientales exigidos por la normativa vigente, garantizando así la seguridad tanto de sus ocupantes como del resto de usuarios de la vía pública. Las estaciones de ITV son empresas especializadas que realizan estas revisiones y gestionan el registro de sus resultados, lo que implica manejar un volumen considerable de información sobre vehículos, clientes e inspecciones.

El presente Trabajo de Fin de Ciclo recoge el desarrollo de ITV CyL, una aplicación de escritorio orientada a la gestión de inspecciones técnicas de vehículos en el ámbito de Castilla y León. Esto se debe a que desde el principio teníamos claro que queríamos desarrollar algo relacionado con vehículos y datos. El punto de partida fue la base de datos pública que publica mensualmente la Dirección General de Tráfico con todos los vehículos matriculados en España, un conjunto de datos que nos ofrecía una base sólida sobre la que construir. A partir de ahí estuvimos barajando varias ideas: un estudio del parque automovilístico español, una aplicación de gestión de multas, un concesionario con taller integrado... Finalmente nos decidimos por una aplicación de ITV por varias razones. Por un lado, nos permitía trabajar de forma natural con los datos de la DGT, que son precisamente datos de vehículos matriculados. Por otro, la naturaleza de una estación de ITV, cuyos trabajadores operan desde puestos fijos, encaja perfectamente con una aplicación de escritorio, que era el tipo de interfaz que queríamos desarrollar desde el principio. Y por último, el proyecto nos permitía cumplir todos los requisitos académicos del ciclo, especialmente en lo que respecta al trabajo con bases de datos relacionales, que en este caso cobran un papel central, tanto en el *backend* como en el *frontend*.

El proyecto surge como ejercicio práctico de integración de tecnologías —*backend* automatizado, base de datos relacional e interfaz gráfica para escritorio— aplicadas a un caso de uso real y cotidiano, centralizando tanto la gestión de los datos del parque vehicular como el registro de las inspecciones realizadas. Para ello, hemos desarrollado una solución que combina un *backend* automatizado en Python, desplegado en *PythonAnywhere*, con una aplicación de escritorio desarrollada en *JavaFX* que actúa como interfaz de usuario.

El *backend* se encarga de descargar mensualmente de forma automática los datos del parque de vehículos publicados por la DGT, filtrar los correspondientes a las nueve provincias de Castilla y León, asignar a cada vehículo una matrícula única generada mediante funciones de hash criptográfico, y volcar toda la información procesada en una base de datos MySQL alojada en *PythonAnywhere*. Este proceso se ejecuta de forma desatendida los días 5 y 20 de cada mes, garantizando que los datos del sistema estén siempre actualizados con la última información publicada por la DGT.

El *frontend*, desarrollado en *JavaFX*, permite a los usuarios del sistema acceder a la aplicación con sus credenciales y operar en función de su rol. Los técnicos pueden iniciar el proceso de revisión de un vehículo introduciendo su matrícula, lo que desencadena una consulta contra la base de datos que recupera los datos precargados del vehículo y, si procede, crea o vincula el registro del cliente correspondiente. A continuación, el técnico cumplimenta los datos propios

de la inspección, que quedan registrados en la base de datos asociados a dicha matrícula. Los administradores, además de las funcionalidades de los técnicos, tienen acceso al área de administración de usuarios, donde pueden dar de alta y de baja a los miembros del equipo.

A lo largo de este documento describimos en detalle las fases de análisis y diseño del sistema, la arquitectura técnica desarrollada, las pruebas realizadas y las conclusiones obtenidas tras el proceso de desarrollo.

2. FASE DE ANÁLISIS

En este apartado se recoge el análisis previo al desarrollo de ITV CyL. En primer lugar, se describen los actores que interactúan con el sistema y sus roles. A continuación, se especifican los requisitos del proyecto, tanto funcionales como no funcionales, que han servido de base para las decisiones de diseño y desarrollo tomadas a lo largo del trabajo.

2.1 Descripción de los actores y roles

El sistema contempla dos tipos de usuarios, ambos pertenecientes a la empresa que opera la estación de ITV:

- El técnico es el usuario operativo del sistema. Su función principal es llevar a cabo las inspecciones técnicas de vehículos, registrar sus resultados y consultar sus propios datos personales. No tiene acceso a la gestión de otros usuarios.
- El administrador dispone de acceso a la gestión completa de usuarios del sistema: puede consultar el listado de usuarios registrados y dar de alta, modificar y dar de baja a los miembros del equipo.

Ambos roles requieren autenticación previa para acceder a cualquier funcionalidad de la aplicación.

2.2 Especificación de requisitos

Los requisitos se han establecido a partir del análisis del caso de uso y de los objetivos del proyecto. Para fijar las prioridades se ha utilizado una escala de tres niveles: Alta, para los requisitos imprescindibles sin los cuales el sistema no puede funcionar correctamente; Media, para los requisitos importantes pero que no bloquean el funcionamiento básico; y Baja, para los requisitos deseables que pueden abordarse en fases posteriores del desarrollo.

2.2.1 Requisitos Funcionales

Los requisitos funcionales describen las capacidades y comportamientos que el sistema debe ofrecer. Se organizan siguiendo el flujo natural del proyecto: primero los relacionados con el *backend* y el procesamiento de datos de la DGT, y después los relacionados con la aplicación de escritorio.

ID	Nombre	Descripción	Prioridad
RF1	Servidor en la nube	Disponer de una cuenta activa en <i>PythonAnywhere</i> que actúe como servidor remoto para ejecutar los scripts del sistema.	Alta
RF2	Base de datos en la nube	Crear y configurar una base de datos MySQL en <i>PythonAnywhere</i> accesible desde la aplicación de escritorio.	Alta

RF3	Descarga automática de datos DGT	Automatizar la descarga mensual de los archivos ZIP publicados por la DGT en su página oficial, detectando siempre la fecha más reciente disponible.	Alta
RF4	Filtrado por provincias de CyL	Extraer del ZIP descargado únicamente los TXT correspondientes a las nueve provincias de Castilla y León.	Alta
RF5	Procesamiento de archivos	Procesar los TXT extraídos, limpiar los valores incorrectos o vacíos, y generar un CSV por provincia con los campos seleccionados.	Alta
RF6	Unicidad de matrículas	Cada vehículo del sistema debe tener una matrícula única e irrepetible, generada de forma determinista a partir de sus datos técnicos mediante funciones de hash criptográfico, distinguiendo entre formato moderno y antiguo según el año de primera matriculación.	Alta
RF7	Control de colisiones de matrícula	Garantizar que no existen matrículas duplicadas en el sistema. En caso de colisión, añadir un sufijo numérico a la matrícula y marcarla como Descartada en la base de datos.	Alta
RF8	Insertión en base de datos	Insertar los datos procesados en la base de datos, gestionando altas, duplicados y bajas de forma incremental mes a mes.	Alta
RF9	Detección automática de bajas	Detectar los vehículos que ya no aparecen en los datos de la DGT y actualizar su estado a Baja en la base de datos.	Alta
RF10	Script coordinador	Disponer de un script principal (coordinador.py) que orqueste la ejecución secuencial de todos los scripts del sistema, deteniéndose ante cualquier error.	Alta
RF11	Automatización mensual	Configurar una tarea programada en <i>PythonAnywhere (Scheduled Tasks)</i> que ejecute el coordinador de forma automática, actuando únicamente en las fechas establecidas de cada mes.	Alta
RF12	Ejecución manual forzada	Permitir la ejecución manual del coordinador en cualquier momento mediante un parámetro de línea de comandos, ignorando el control de fechas.	Media
RF13	Modelo de datos	El sistema debe disponer de las tablas necesarias para gestionar la información de vehículos, usuarios, clientes e inspecciones, con los campos adecuados para cada entidad y las relaciones entre ellas.	Alta

RF14	Autenticación	La aplicación debe mostrar una pantalla de <i>login</i> como primera vista al arrancar y requerir credenciales válidas en todos los casos para acceder al sistema.	Alta
RF15	Gestión de usuarios	La aplicación debe permitir al administrador consultar el listado de usuarios registrados y dar de alta, modificar (nombre, teléfono, correo y rol) y dar de baja usuarios del sistema. El técnico no debe tener acceso a esta funcionalidad.	Alta
RF16	Revisión de vehículo	La aplicación debe permitir iniciar el proceso de inspección introduciendo la matrícula del vehículo, recuperando automáticamente sus datos precargados desde la base de datos.	Alta
RF17	Gestión de clientes	Al iniciar una inspección, el sistema debe comprobar si la matrícula tiene un cliente asociado. Si lo tiene, carga sus datos; si no, permite crear un cliente nuevo.	Alta
RF18	Registro de inspección	La aplicación debe guiar al técnico a través de las distintas fases de la inspección mediante pantallas sucesivas. Al finalizar, el técnico debe poder indicar el resultado y emitir un informe en PDF con todos los datos del vehículo, el cliente, los fallos detectados y las observaciones realizadas.	Alta
RF19	Consistencia mensual	El sistema debe producir los mismos resultados ante los mismos datos de entrada, garantizando que un mismo vehículo recibe siempre la misma matrícula en cada ejecución mensual.	Alta
RF20	Gestión de errores	Ante cualquier fallo en el proceso, el sistema debe informar del error con el mayor detalle posible y detener la ejecución de forma controlada, sin dejar datos incompletos en la base de datos.	Alta

2.2.2 Requisitos no funcionales

Los requisitos no funcionales definen las cualidades y restricciones que debe cumplir el sistema con independencia de las funcionalidades concretas que ofrece

ID	Nombre	Descripción	Prioridad
RNF1	Rendimiento	El sistema debe ser capaz de procesar y gestionar cerca de dos millones de registros mensuales en un tiempo razonable, sin comprometer la estabilidad del servidor.	Alta

RNF2	Disponibilidad del servicio	El sistema depende de servicios externos como <i>PythonAnywhere</i> y la página de la DGT. Aunque su disponibilidad no está bajo nuestro control, el sistema debe gestionar correctamente los fallos de conexión e informar de ellos sin corromperse.	Alta
RNF3	Control de acceso por roles	El acceso a la aplicación debe requerir autenticación obligatoria.	Alta
RNF4	Integridad de los datos	El sistema debe garantizar que los datos no se pierden ni se corrompen durante el proceso de inserción. Todas las operaciones sobre la base de datos deben realizarse de forma atómica, deshaciendo los cambios si se produce cualquier error.	Alta
RNF5	Trazabilidad del proceso	El sistema debe informar por pantalla del estado de cada paso del proceso (descarga, extracción, procesado e inserción), facilitando la detección de errores.	Media
RNF6	Mantenibilidad	El código debe estar estructurado en scripts independientes con responsabilidades bien definidas, comentado y documentado para facilitar su mantenimiento y evolución.	Alta
RNF7	Escalabilidad	El sistema debe estar diseñado para poder incorporar nuevas provincias o ampliar el conjunto de datos procesados sin necesidad de reescribir el código.	Media
RNF8	Usabilidad	La interfaz de usuario debe ser clara e intuitiva, permitiendo que cualquier usuario pueda operar el sistema sin necesidad de formación técnica específica.	Alta
RNF9	Almacenamiento	La base de datos MySQL debe ser capaz de almacenar el volumen de datos generado (estimado en torno a 500-600 MB) dentro de los límites del plan contratado en <i>PythonAnywhere</i> (5 GB).	Alta
RNF10	Codificación de caracteres	Todos los ficheros generados (CSVs) deben usar codificación UTF-8 con BOM para garantizar la correcta visualización de caracteres especiales como tildes y eñes en cualquier entorno.	Media
RNF11	Diseño de la interfaz	La aplicación debe presentar una interfaz visualmente coherente, con una estructura clara y un estilo consistente en todas sus pantallas.	Media

RNF12	Manual de usuario	El proyecto debe incluir documentación de uso dirigida a los usuarios del sistema, que les permita operar la aplicación sin conocimientos técnicos previos.	Media
-------	-------------------	---	-------

3. FASE DE DISEÑO

La fase de diseño recoge las decisiones técnicas y arquitectónicas que dan forma a ITV CyL y explica cómo se ha planificado su desarrollo antes de pasar a la implementación.

La aplicación permite gestionar el proceso completo de una inspección técnica de vehículos: desde el acceso al sistema mediante autenticación por roles, pasando por la consulta de datos del vehículo y del cliente, el registro de cada fase de la inspección y la emisión del informe final en PDF, hasta la gestión de los usuarios del sistema por parte del administrador. Todo ello respaldado por un *backend* automatizado que mantiene actualizada la base de datos de vehículos con los datos publicados mensualmente por la DGT.

Para llevarlo a cabo, el sistema se articula en torno a tres elementos principales. En primer lugar, los casos de uso definen las interacciones entre los actores del sistema y las funcionalidades que estos pueden realizar según su rol. En segundo lugar, el análisis de persistencia establece cómo y dónde se almacena la información, optando por una base de datos relacional MySQL cuyo modelo de datos se recoge en el diagrama entidad-relación. En tercer lugar, el diseño funcional describe la arquitectura del sistema, tanto el *backend* automatizado en Python como la aplicación de escritorio en *JavaFX*, detallando la lógica de cada componente y las decisiones técnicas tomadas durante el desarrollo. El apartado se completa con los prototipos de la interfaz, que muestran la evolución del diseño visual desde las primeras ideas hasta la propuesta de media fidelidad.

3.1 Casos de uso

Los casos de uso describen las interacciones entre los actores del sistema y las funcionalidades que este les ofrece. En nuestra aplicación, intervienen dos actores diferenciados: el técnico y el administrador. Ambos acceden al sistema desde la misma aplicación de escritorio, pero con permisos y flujos de trabajo distintos según su rol.

El técnico es el usuario operativo del sistema. Su flujo comienza siempre con el inicio de sesión, tras el cual accede directamente a la funcionalidad principal: realizar una inspección. El proceso arranca introduciendo la matrícula del vehículo, que el sistema consulta contra la base de datos para recuperar sus datos precargados. En ese momento el sistema también comprueba si existe un cliente asociado a esa matrícula: si lo hay, carga sus datos automáticamente; si no, habilita la opción de añadir un cliente nuevo. A continuación el técnico recorre de forma secuencial las distintas fases de la inspección —documentación, acondicionamiento exterior, chasis y carrocería, acondicionamiento interior, alumbrado y señalización, emisiones contaminantes, frenos, dirección, ejes, ruedas y suspensión, y motor y transmisión— hasta llegar a la pantalla final, donde indica el resultado de la revisión y puede emitir un informe en PDF con todos los datos del vehículo, el cliente, los fallos detectados y las observaciones registradas a lo largo del proceso.

El administrador, tras iniciar sesión, accede a la gestión de usuarios del sistema. Desde una pantalla con el listado completo de usuarios registrados puede dar de alta nuevos técnicos o

administradores, modificar los datos de cualquier usuario existente y dar de baja a quienes ya no formen parte del equipo.

https://drive.google.com/file/d/1NO0G-Tw2CUMEFz0apDPU2GEf-isuYHJh/view?usp=drive_link

3.2 Análisis de persistencia

Para el almacenamiento de los datos del sistema hemos optado por una base de datos relacional MySQL alojada en *PythonAnywhere*, la misma plataforma que utilizamos para ejecutar los scripts del *backend*. Esta decisión responde a varias razones.

La primera es la integración natural con el resto del sistema. Al tener el *backend* y la base de datos en la misma plataforma, la comunicación entre los scripts de Python y MySQL es directa y eficiente, sin necesidad de configurar conexiones externas ni gestionar infraestructuras adicionales.

La segunda es que los datos que manejamos son claramente relacionales. Los vehículos se asocian a clientes, los clientes a inspecciones, las inspecciones a sus defectos y los usuarios al sistema de acceso. Este tipo de relaciones entre entidades se modela de forma natural y robusta en una base de datos relacional, con claves foráneas que garantizan la integridad referencial.

La tercera es la fiabilidad y el soporte de MySQL. Es el sistema gestor de bases de datos relacionales más extendido en entornos profesionales, con una documentación amplia y una compatibilidad total tanto con Python (a través de `MySQL-connector-python`) como con Java (a través de `JDBC`), los dos lenguajes que usamos en el proyecto. Así mismo, es el gestor que más hemos utilizado en clase y para nosotros era lo más lógico.

El modelo de datos se compone de seis tablas. La tabla 'vehiculos_cyl' almacena los datos del parque de vehículos procesados mensualmente desde los ficheros de la DGT, con la matrícula generada como clave primaria. La tabla clientes recoge los datos de los clientes que acuden a la estación. La tabla 'vehiculo_cliente' actúa como tabla intermedia para modelar la relación N:M entre vehículos y clientes, ya que un mismo vehículo puede ser traído a inspeccionar por distintas personas a lo largo del tiempo. La tabla 'inspecciones' registra cada revisión técnica realizada, asociada a la matrícula del vehículo. La tabla 'inspeccion_defectos' almacena los defectos concretos detectados en cada inspección. Y la tabla 'usuarios' gestiona el acceso al sistema con sus dos roles: técnico y administrador. La tabla 'tarifas', independiente del resto, almacena los precios de la inspección según el tipo de vehículo y se consulta directamente desde la aplicación.

3.2.1 Diagrama Entidad-Relación

https://drive.google.com/file/d/1bxMMwflPjd7UTL9xVXwaupCjmkztgmOK/view?usp=drive_link

https://drive.google.com/file/d/15YvNp_Pxfs8vOANHbpjnUVo3kj_cHdnB/view?usp=drive_link

3.3 Diseño funcional y lógica del sistema

En este apartado describimos las decisiones técnicas y arquitectónicas que han dado forma al sistema. Explicamos cómo está estructurado, qué tecnologías hemos utilizado y por qué, qué problemas encontramos durante el desarrollo y cómo los resolvimos, y cómo está organizado el código en scripts con responsabilidades bien definidas.

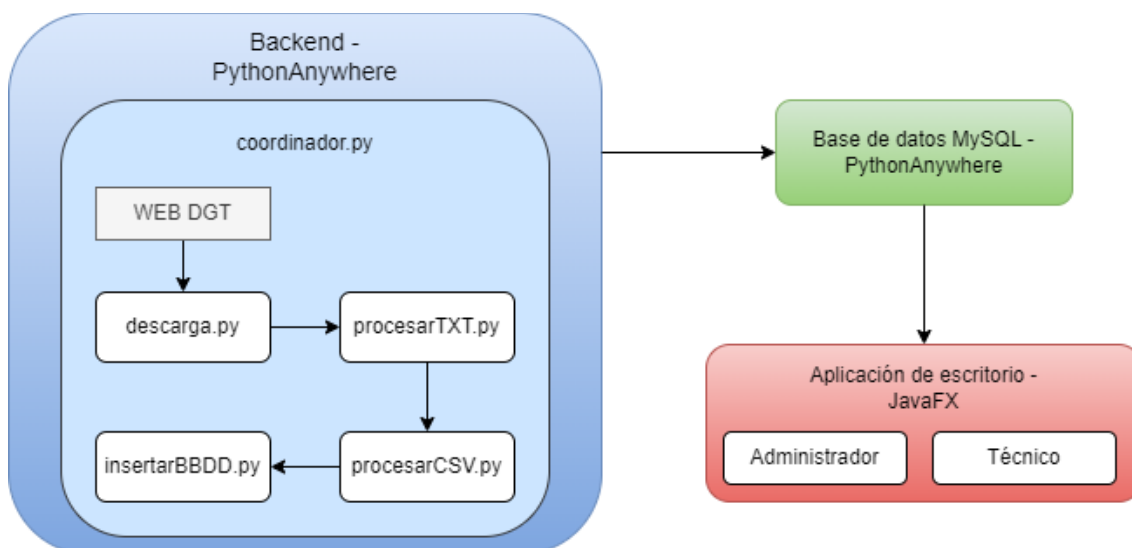
3.3.1 Arquitectura del sistema

El sistema se estructura en tres capas bien diferenciadas que trabajan de forma coordinada: el *backend* automatizado, la base de datos y la aplicación de escritorio.

El *backend*, alojado en *PythonAnywhere*, es el encargado de todo el trabajo con los datos de la DGT. Cinco scripts de Python se ejecutan en cadena cada mes: el primero descarga el fichero de datos de la web oficial de la DGT, el segundo extrae únicamente los datos de las provincias de Castilla y León, el tercero los limpia y genera las matrículas únicas, el cuarto sube todo a la base de datos, y el quinto —el coordinador— orquesta a los otros cuatro y se encarga de que el proceso se lance automáticamente los días 5 y 20 de cada mes.

La base de datos MySQL, también en *PythonAnywhere*, almacena la información de los vehículos, los usuarios del sistema, los clientes y las inspecciones realizadas. Actúa como punto de unión entre el *backend* y el *frontend*.

La aplicación de escritorio, desarrollada en *JavaFX*, es la herramienta que usan los trabajadores en su día a día. Se conecta a la base de datos para consultar los datos de los vehículos, registrar inspecciones y gestionar los usuarios del sistema.



Elección de tecnologías

La elección de Python para el *backend* fue una decisión clara desde el principio. Frente a Java, que era la otra opción que manejábamos por ser el lenguaje principal del ciclo, Python ofrece una experiencia de trabajo mucho más ágil cuando se trata de procesar y transformar datos: librerías como *pandas*, *requests* o *BeautifulSoup* permiten resolver en pocas líneas tareas

que en Java requerirían un desarrollo considerablemente más extenso. Dado que el núcleo del *backend* es precisamente el procesamiento de grandes volúmenes de datos, Python era la elección natural.

En cuanto a la plataforma de alojamiento, nuestra tutora Aida nos recomendó *PythonAnywhere* por su integración nativa con Python y por la facilidad con la que permite desplegar scripts, configurar tareas programadas y gestionar una base de datos MySQL desde un único entorno. Esta recomendación resultó acertada: la plataforma nos permitió tener el *backend* funcionando en producción de forma sencilla y sin necesidad de gestionar infraestructura propia.

Para la base de datos elegimos MySQL principalmente por su compatibilidad con *PythonAnywhere*, que lo ofrece de forma integrada en sus planes, y por ser el sistema gestor de bases de datos relacional más extendido en entornos profesionales. Su uso con Java desde el *frontend* mediante JDBC es también bien conocido y está ampliamente documentado.

Problemas encontrados y soluciones

El desarrollo del sistema no estuvo exento de dificultades. A continuación, describimos los principales problemas que encontramos y cómo los resolvimos, aunque cada uno de ellos se desarrolla con más detalle en la sección correspondiente de cada script.

El primer problema fue el anonimato de los datos. Los ficheros de la DGT no contienen ningún identificador único por vehículo —solo características técnicas— por lo que no había forma directa de distinguir dos coches idénticos ni de asignarles un identificador estable. Lo resolvimos mediante un sistema de doble hash: el primero identifica la combinación de características técnicas y el segundo, combinado con un número de instancia, identifica a cada vehículo de forma individual y reproducible.

El segundo problema fue el volumen de datos. Manejar cerca de dos millones de registros mensuales impone restricciones tanto en memoria como en almacenamiento. Lo resolvimos extrayendo del ZIP de la DGT únicamente los ficheros de las provincias que nos interesan, en lugar de descomprimirlo completo, y eliminando el ZIP una vez terminada la extracción para liberar espacio en disco.

El tercer problema fue el rechazo de la DGT a nuestras conexiones. La versión gratuita de *PythonAnywhere* enruta todas las conexiones a través de un proxy compartido, y la página de la DGT bloqueaba las peticiones que provenían de ese proxy, impidiendo la descarga automática de los datos. Lo resolvimos migrando al plan de pago de la plataforma, que permite conexiones directas a internet sin restricciones de proxy.

Relacionado con lo anterior, la versión gratuita también imponía un límite de 512 MB para la base de datos, insuficiente para el volumen de datos que manejamos. El plan de pago amplió ese límite a 5 GB, lo que nos da margen más que suficiente para las necesidades actuales del proyecto.

Por último, durante el desarrollo detectamos que la provincia de Ávila no aparece de forma consistente en los ficheros publicados por la DGT, que trata de forma irregular los nombres de provincia que contienen tildes. Resolvimos este problema aplicando una normalización de

caracteres antes de comparar los nombres de los ficheros, de forma que el sistema pueda procesar Ávila cuando aparece y continuar sin ella cuando no lo hace, sin necesidad de modificar el código.

3.3.2 Scripts desarrollados

El *backend* está compuesto por cinco scripts de Python, cada uno con una responsabilidad única y bien definida. Esta división no es arbitraria: responde a un principio de diseño que facilita el mantenimiento, la depuración y la evolución del sistema. Si algo falla, es inmediatamente localizable en el script responsable. Si queremos modificar una parte del proceso, podemos hacerlo sin tocar el resto.

Los cuatro primeros scripts forman una cadena de procesado: *descarga.py* obtiene los datos de la DGT, *procesarTXT.py* los filtra y los extrae, *procesarCSV.py* los transforma y genera las matrículas, e *insertarBBDD.py* los sube a la base de datos. El quinto script, *coordinador.py*, actúa como punto de entrada único que orquesta a los otros cuatro y gestiona la automatización mensual.

Descarga.py

El primer script del pipeline tiene una única responsabilidad: detectar y descargar el archivo ZIP más reciente publicado por la DGT con los datos del parque de vehículos mensual.

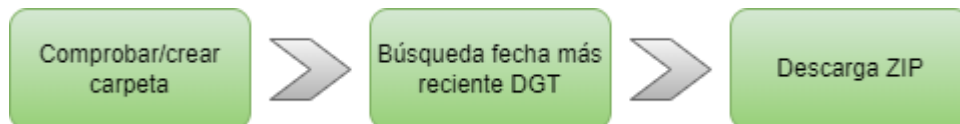
Para llevar a cabo su función hace uso de cuatro librerías: *os* para trabajar con rutas y carpetas del sistema, *re* para buscar patrones de fecha dentro del HTML de la DGT mediante expresiones regulares, *requests* para realizar la descarga tanto de la página web como del archivo ZIP, y *BeautifulSoup* para interpretar el HTML y extraer el texto visible de forma sencilla.

Al inicio del script se centralizan en constantes la carpeta de destino de los ZIPs, la dirección oficial de la página de la DGT y el patrón de la URL del archivo ZIP, con un marcador de fecha que se sustituye automáticamente en cada ejecución.

El núcleo del script es la función que detecta la fecha más reciente publicada por la DGT. En lugar de que alguien la introduzca manualmente, el script descarga la página oficial, extrae su texto con *BeautifulSoup* y aplica una expresión regular para encontrar todas las cadenas con formato YYYYMM. Con ese formato, la comparación alfabética coincide exactamente con la comparación cronológica, por lo que basta con quedarse con el valor máximo para obtener siempre la fecha más reciente. Con esa fecha construye la URL exacta del ZIP y lo descarga en modo binario, byte a byte, sin ninguna transformación de codificación, ya que se trata de un archivo comprimido y no de texto plano.

El script incluye además una función que garantiza que la carpeta de destino existe antes de intentar guardar el fichero, evitando errores en rutas inexistentes. Como en el resto de scripts, toda la lógica se centraliza en una función `main()` que ejecuta los pasos en el orden correcto. El bloque `if __name__ == "__main__"` que la invoca tiene un propósito concreto: cuando Python ejecuta un fichero directamente le asigna el nombre especial `__main__`, pero cuando ese fichero es importado desde otro script recibe su propio nombre. Gracias a esa distinción, el

coordinador puede importar las funciones de *descarga.py* para usarlas sin que el script se ejecute automáticamente y lance una descarga no solicitada.



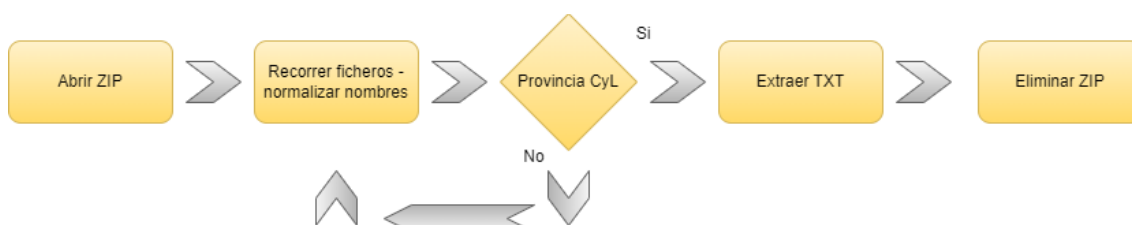
ProcesaTXT.py

El segundo script del pipeline abre el ZIP descargado, extrae únicamente los ficheros TXT de las provincias de Castilla y León y elimina el ZIP una vez terminado el proceso para liberar espacio en disco.

Para trabajar con archivos comprimidos utiliza la librería *zipfile*. Importa además la función de detección de fecha de *descarga.py*, lo que le permite saber qué ZIP debe procesar sin que sea necesario indicarlo manualmente.

El núcleo del script es la función que recorre todos los ficheros contenidos en el ZIP y extrae solo los que corresponden a nuestras provincias. Para que la comparación sea robusta independientemente de cómo haya nombrado la DGT los ficheros ese mes, aplica una normalización de caracteres con *unicodedata* antes de comparar, eliminando tildes y convirtiendo todo a mayúsculas. Esto resuelve el problema de que Ávila, León o cualquier otra provincia con tilde pueda aparecer con variantes distintas según el mes.

Para evitar extraer el mismo fichero más de una vez, el script mantiene un conjunto de nombres ya procesados que consulta antes de cada extracción. Una vez terminado, comprueba si el ZIP sigue existiendo en disco y lo elimina, conservando únicamente los TXT necesarios para el siguiente paso.



ProcesaCSV.py

El tercer script del pipeline es el más complejo del proyecto. Su función es leer los ficheros TXT extraídos, procesar sus datos y generar nueve archivos CSV, uno por cada provincia, listos para ser insertados en la base de datos. La decisión de generar un fichero por provincia en lugar de uno único con todos los datos responde a una limitación práctica que detectamos durante el desarrollo: al intentar trabajar con un único CSV que agrupara todos los registros, la carga era excesiva para Excel, que terminaba dando error. Dividir los datos por provincia nos permite además localizar y gestionar los errores de forma mucho más controlada, sabiendo exactamente en qué provincia se ha producido el problema.

El primer desafío que plantea este script es uno de naturaleza estructural: los ficheros de la DGT no contienen ningún identificador único por vehículo. No hay ninguna clave primaria, ningún número de bastidor, ningún dato que permita distinguir un registro de otro de forma directa.

Los datos son completamente anónimos desde el punto de vista identificativo. Esto significa que necesitábamos crear nosotros ese identificador, y que además tenía que ser estable, es decir, que el mismo vehículo produjera siempre el mismo identificador en cada ejecución mensual. La matrícula que generamos cumple exactamente esa función: actúa como clave primaria artificial del sistema.

Para construir ese identificador partimos de los propios datos técnicos del vehículo que nos proporciona la DGT. De todos los campos disponibles, seleccionamos aquellos que describen las características físicas e inmutables del vehículo: datos relacionados con su construcción, mecánica, dimensiones y homologación. Excluimos deliberadamente todos los campos que reflejan situaciones administrativas o circunstanciales, como el tipo de titular, el número de propietarios o la fecha de matriculación actual, porque esos datos pueden cambiar a lo largo de la vida del vehículo sin que el vehículo en sí haya cambiado. El criterio de selección fue por tanto claro: solo lo que describe al vehículo de forma permanente.

El reto central de este script es asignar a cada vehículo una matrícula única, que lo identifique de forma inequívoca, que sea la misma en cada ejecución mensual y que no dependa de ningún dato personal. Para ello adoptamos un sistema de doble hash¹ basado en el algoritmo MD5. El primer hash se calcula a partir de los datos técnicos seleccionados y sirve para identificar el tipo de vehículo: dos coches con exactamente las mismas características técnicas producirán el mismo primer hash. El segundo hash se calcula combinando el primero con un número de instancia, lo que garantiza que dos vehículos con características idénticas reciban matrículas distintas. Este segundo hash es siempre único para cada registro, ya que nunca dos vehículos pueden compartir el mismo primer hash y el mismo número de instancia simultáneamente.

Con ese segundo hash generamos la matrícula. Para los vehículos matriculados desde el año 2000 usamos el formato actual de cuatro dígitos y tres letras; para los anteriores, el formato antiguo en el que las dos primeras letras identifican la provincia. Los dos formatos usan alfabetos distintos que reflejan la realidad histórica de las matrículas españolas: las modernas usan un alfabeto de 21 consonantes, excluyendo la Ñ y las vocales para evitar combinaciones que puedan resultar inapropiadas, mientras que las antiguas usan un alfabeto de 25 letras que sí incluye las vocales, siendo más fiel al formato que se usó en España antes del año 2000.

Aun así, existe una probabilidad matemática de que dos vehículos distintos generen la misma combinación de dígitos y letras, lo que se conoce como colisión². Para gestionirlas, el script

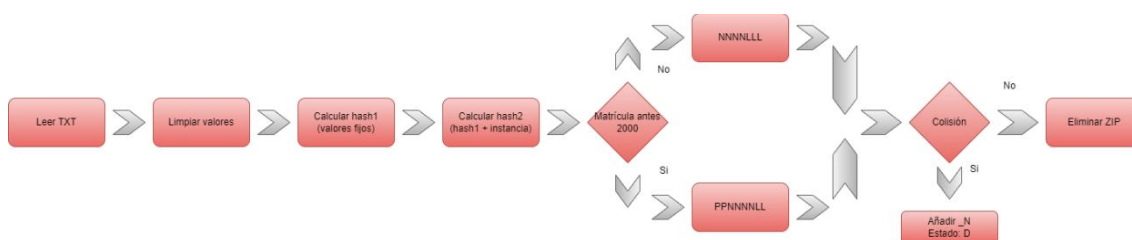
¹ Un hash es una función matemática que transforma cualquier conjunto de datos (texto, archivos, contraseñas) en una serie alfanumérica única de longitud fija, actuando como una "huella digital" inmutable.

² Para entender por qué ocurre, basta con comparar el número de vehículos que manejamos con el número de combinaciones posibles. Las matrículas modernas, con cuatro dígitos y tres consonantes, ofrecen un espacio de 92.610.000 combinaciones posibles; las antiguas, con las letras de provincia fijas y dos letras del alfabeto completo de 25 letras (excluyendo la 'Ñ'), suman aproximadamente 50.000.000 combinaciones adicionales repartidas entre las ocho provincias. Cabe recordar que todos estos cálculos se realizan sobre los datos de ocho provincias de Castilla y León, ya que Ávila no aparece de forma consistente en los archivos de la DGT. Con 1.875.073 registros procesados y aplicando la paradoja del cumpleaños — que establece que la probabilidad de colisión crece de forma cuadrática con el número de elementos — el modelo predice unas 15.750 colisiones, lo que representaría aproximadamente un 0,84%

mantiene un mapa global de matrículas ya asignadas: si una matrícula ya existe, a la nueva se le añade un sufijo numérico y se marca como Descartada mediante el campo ESTADO_ACTUAL, que puede tomar los valores A para Activa, D para Descartada y B para Baja.

Un problema detectado durante el desarrollo fue que la función `apply()` de pandas, usada inicialmente para recorrer las filas de forma eficiente, no garantiza el orden estricto de ejecución cuando hay efectos secundarios en variables globales como el mapa de instancias. Esto provocaba matrículas duplicadas sin sufijo. La solución fue revertir al bucle `iterrows()`, que aunque es algo más lento garantiza que el procesado es siempre secuencial y predecible.

Antes de calcular ningún hash, cada valor pasa por una función de limpieza con tres filtros en cascada que normalizan los datos incorrectos, vacíos o con caracteres no imprimibles a un valor neutro, garantizando que dos vehículos con las mismas características siempre produzcan el mismo hash. Una vez generadas las matrículas, el script aplica conversiones de legibilidad sobre los campos codificados — provincia, procedencia y nuevo/usado — y añade la fecha y hora de ejecución como campo de auditoría. Los CSVs se guardan con codificación UTF-8 con BOM para que Excel los abra correctamente.



InsertarBBDD.py

El cuarto script lee los nueve CSVs generados e inserta su contenido en la base de datos MySQL de forma incremental y segura mes a mes. Utiliza la librería `MySQL-connector-python` para conectarse a MySQL.

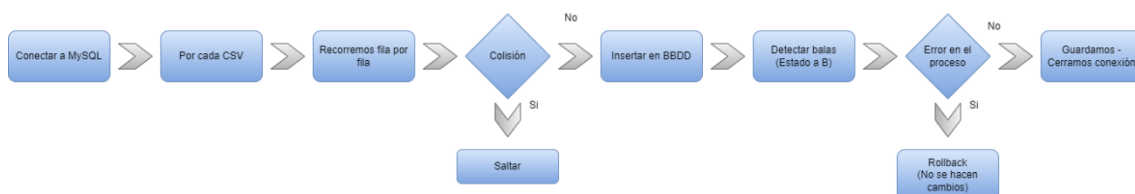
Antes de procesar ningún CSV, carga en memoria todas las matrículas existentes en la base de datos mediante una consulta `SELECT`, almacenándolas en un conjunto de Python para poder hacer búsquedas instantáneas independientemente del volumen. Para cada fila de cada CSV, el script compara la matrícula con ese conjunto: si ya existe, la salta; si no, la inserta. Usa `INSERT IGNORE` como medida de seguridad adicional para que cualquier duplicado inesperado sea descartado silenciosamente sin abortar el proceso.

Al mismo tiempo acumula todas las matrículas de los CSVs en un segundo conjunto. Al terminar, la resta entre el conjunto de matrículas de la base de datos y el de los CSVs del mes actual identifica los vehículos que ya no aparecen en los datos de la DGT, que se marcan como Baja mediante `UPDATE`. Para evitar superar el límite de tamaño de paquete de MySQL, las bajas no se procesan en una única consulta sino en lotes de 1.000 matrículas, lo que garantiza que la operación funciona correctamente independientemente del volumen de bajas detectadas.

del total. Los datos reales obtenidos arrojan 14.069 matrículas descartadas sobre un total de 1.875.073 registros, lo que supone un 0,75% del total, con 1.861.004 matrículas activas. Un resultado que se ajusta con relativa precisión a la predicción teórica.

Todas las operaciones se realizan dentro de una transacción: si el proceso termina sin errores se confirman con `commit`; si falla en cualquier punto se deshacen con `rollback`, garantizando que la base de datos nunca queda en un estado inconsistente. La conexión se cierra siempre al final gracias al bloque `finally`.

Un aspecto que requirió atención especial fue la conversión de fechas: los CSVs usan el formato español DD/MM/YYYY mientras que MySQL espera YYYY-MM-DD para los campos de tipo DATE y con hora para los de tipo DATETIME. Se implementó una función de conversión que acepta ambos formatos y devuelve el valor correcto, o nulo si el valor es inválido.



Coordinador.py

El último script del proyecto no procesa ningún dato por sí mismo: importa y ejecuta en orden los cuatro scripts anteriores, convirtiendo todo el pipeline en un proceso que puede lanzarse con un único comando.

Cada script se ejecuta dentro de una función que envuelve la llamada en un bloque de control de errores. Si el script completa su ejecución correctamente el coordinador pasa al siguiente; si falla, muestra el nombre del paso y el motivo del error y detiene el proceso de inmediato. Los cuatro scripts son un todo indivisible — no tiene sentido ejecutar uno sin los anteriores — por lo que ante cualquier fallo lo correcto es parar y notificar.

Para la automatización mensual el coordinador se configura como tarea diaria en *PythonAnywhere* a las tres de la madrugada. Sin embargo, al inicio de cada ejecución comprueba si el día actual es el 5 o el 20 del mes — las fechas en las que la DGT suele haber publicado los datos nuevos — y solo actúa si lo es.

Con ese argumento el script omite la comprobación del día y ejecuta el pipeline completo independientemente de la fecha. La tarea programada de *PythonAnywhere* nunca incluye ese argumento, por lo que respeta siempre el control de días de forma automática.

3.4 Análisis diseño interfaz

Este apartado recoge las decisiones de diseño tomadas para la interfaz de usuario de ITV CyL, la justificación de las tecnologías elegidas y la arquitectura seguida en el desarrollo del *frontend*.

3.4.1 Elección de JAVAFX y herramientas de desarrollo

Para el desarrollo de la interfaz de usuario elegimos *JavaFX* como tecnología principal. Durante el ciclo formativo habíamos trabajado con Apache NetBeans para crear aplicaciones de escritorio, y nuestras tutoras nos orientaron hacia *JavaFX* como una alternativa más moderna y potente que mantiene una curva de aprendizaje razonable para quien ya conoce el entorno de

escritorio en Java. Esta recomendación nos permitió desarrollar una interfaz más cuidada y moderna que la que habríamos conseguido con las herramientas que ya conocíamos.

Para el diseño visual de las pantallas utilizamos *Scene Builder*, una herramienta que permite construir interfaces gráficas de forma visual mediante arrastrar y soltar componentes (*drag and drop*), generando automáticamente los ficheros FXML que luego se integran en el proyecto. Para la parte funcional, el código Java que da vida a cada pantalla, usamos Visual Studio Code como entorno de desarrollo, con la extensión de Java y las librerías de *JavaFX* configuradas manualmente.

Todas las pantallas de la aplicación parten de un panel base sobre el que se combinan distintos tipos de contenedores según las necesidades de cada vista. Cada pantalla tiene una imagen de fondo asignada mediante un componente *ImageView*, lo que le da a la aplicación una apariencia coherente y profesional. Los estilos visuales se aplican mediante propiedades CSS con el prefijo *-fx*. Cada componente tiene asignado un identificador mediante el atributo *fx:id*, que permite referenciarlo desde el código Java del controlador correspondiente, y cada ventana tiene asociada una clase controladora mediante la propiedad *Controller* de *Scene Builder*.

3.4.2 Conexión con la base de datos

La conexión entre la aplicación *JavaFX* y la base de datos MySQL alojada en *PythonAnywhere* presentó una dificultad técnica inicial: *PythonAnywhere* no permite conexiones directas al puerto estándar de MySQL (3306) desde el exterior por razones de seguridad. Para resolverlo seguimos las instrucciones oficiales de la plataforma, que explican cómo establecer la conexión de forma correcta desde una aplicación externa. La lógica de conexión está centralizada en un único fichero dentro del paquete *CONNECTION* del proyecto, lo que facilita cambiar las credenciales o el host sin tener que modificar el resto del código.

3.4.3 Arquitectura del frontend

Para estructurar el proyecto hemos seguido el patrón de diseño DAO (Data Access Object), que es una de las buenas prácticas más extendidas en el desarrollo de aplicaciones con acceso a base de datos. Este patrón separa de forma clara la lógica de acceso a datos del resto de la aplicación, lo que facilita el mantenimiento, las pruebas y la evolución del código. Si en el futuro se decide cambiar la base de datos o modificar alguna consulta, los cambios quedan aislados en la capa DAO sin afectar al resto del sistema.

El proyecto se organiza en los siguientes paquetes dentro de la carpeta *src*:

- *CONNECTION* contiene la clase encargada de establecer y gestionar la conexión con la base de datos de *PythonAnywhere*.
- *DAOS* agrupa las clases DAO del proyecto, una por cada entidad del sistema, que encapsulan todas las operaciones de consulta, inserción, modificación y eliminación sobre la base de datos.

- POJOS contiene las clases POJO (Plain Old Java Object), que son clases simples que representan las entidades del sistema — vehículo, cliente, inspección, usuario — con sus atributos y métodos getter y setter correspondientes.
- CONTROLLERS incluye los controladores de cada ventana, que son las clases Java que gestionan la lógica de cada pantalla: responden a las acciones del usuario, llaman a los DAOs para obtener o guardar datos y actualizan la vista en consecuencia.
- UI contiene las clases correspondientes a las ventanas de la aplicación.
- vistas almacena los ficheros FXML generados con Scene Builder, que definen la estructura visual de cada pantalla.
- img es la carpeta donde se guardan todas las imágenes utilizadas en la aplicación, de forma que sean accesibles cuando el proyecto se exporta.

Para gestionar el flujo de datos durante una inspección, que implica recorrer varias pantallas sucesivas acumulando información, hemos implementado una clase de utilidad que actúa como buffer temporal. Esta clase almacena los datos que el técnico va introduciendo en cada pantalla de la inspección para que estén disponibles al final del proceso, tanto para guardarlos en la base de datos como para generar el informe PDF.

Para la generación del informe PDF utilizamos la librería Thymeleaf, que permite definir una plantilla HTML con variables que se sustituyen en tiempo de ejecución por los datos reales de la inspección, y OpenHTMLtoPDF, que convierte ese HTML a PDF aplicando el CSS definido en la plantilla.

Además de estas, el proyecto incluye otras clases de utilidad para gestionar la sesión del usuario autenticado, controlar las opciones del menú lateral de la inspección y centralizar la lógica de apertura y cierre de ventanas.

3.4.4 Prototipo de Baja Fidelidad

https://drive.google.com/file/d/1a-CZOoZGaaYZzEn80biR0xMz1ORnfgBH/view?usp=drive_link

3.4.5 Prototipo de Media Fidelidad

https://drive.google.com/file/d/1NAM2In281nAfXxE0cOuOIVJEXp9YJbX8/view?usp=drive_link

4. FASE DE PRUEBAS

https://docs.google.com/spreadsheets/d/141c5bJ-P6PtbfUr5KjjJfOOABKTVtpgP/edit?usp=drive_link&ouid=110930095367198373559&rtpof=true&sd=true

5. DESPLIEGUE DE LA APLICACIÓN

Se describirá el proceso seguido para el despliegue del proyecto, de modo que este sea accesible más allá del entorno local. Se detallarán las **herramientas empleadas** y la **relevancia** de cada una de ellas en la ejecución de dicha tarea. Este proceso podrá variar significativamente en función del tipo de proyecto desarrollado.

6. DOCUMENTACIÓN DE LA APLICACIÓN

Dependiendo de la naturaleza del proyecto será necesario adjuntar información acerca de los **requisitos mínimos necesarios para el correcto funcionamiento**, ya sean hardware o software; así como **manual de instalación** y/o **manual de usuario**.

En esta sección se expondrán los requisitos mínimos y se hará mención de los manuales desarrollados y cualquier aspecto general sobre estos que haya que considerar; pero estos se incluirán de forma completa en la sección de “Anexos”.

Nota: En el caso de que la aplicación cuente con algún sistema de ayuda integrado, se debe detallar su funcionamiento también en este apartado.

7. CONCLUSIONES

Este apartado debe dar cierre a todo el trabajo realizado, para ello se analizará el **grado de cumplimiento de los objetivos fijados**, atendiendo a aquellas cosas que han salido como se esperaba y a los problemas que han surgido (en caso de haberse solventado es buena idea explicar cómo se logró solucionar, al menos para aquellos casos más relevantes y/o significativos). Del mismo modo se recogerán **propuestas de trabajo futuro** y/o posibles ampliaciones que se podrían realizar. Por último, es interesante añadir una **valoración personal** en la que se exponga que ha aportado al alumno el desarrollo de este proyecto en todos los ámbitos (personal, académico...), en este momento también se puede incluir cualquier reflexión, relacionada con la formación que este trabajo pretende concluir, que se considere.

8. BIBLIOGRAFÍA

Backend y Python

- Documentación oficial de Python: <https://docs.python.org/3/>
- Documentación oficial de pandas: <https://pandas.pydata.org/docs/>
- Documentación oficial de BeautifulSoup: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
- Documentación oficial del conector: <https://dev.mysql.com/doc/connector-python/en/>
- Documentación oficial de hashlib (Python): <https://docs.python.org/3/library/hashlib.html>
- Stack Overflow (resolución de dudas y errores durante el desarrollo en Python): <https://stackoverflow.com/>
- Reddit (resolución de dudas y errores durante el desarrollo en Python): <https://www.reddit.com/>

PythonAnywhere

- Documentación oficial de PythonAnywhere: <https://help.pythonanywhere.com/>
- Acceso a MySQL desde fuera de PythonAnywhere: <https://help.pythonanywhere.com/pages/AccessingMySQLFromOutsidePythonAnywhere/>
- Vídeo: 11. PythonAnywhere: <https://www.youtube.com/watch?v=IfQFI-bKve4>
- Vídeo: PythonAnywhere — introducción a PythonAnywhere (UNAULA): <https://www.youtube.com/watch?v=bwxvEuKTInU>
- Vídeo: Schedule Python Tasks on PythonAnywhere — Automate Everything with Python: <https://www.youtube.com/watch?v=C6NThuZiLjU>

Base de datos

- Documentación oficial de MySQL: <https://dev.mysql.com/doc/>
- Tarifas ITV Castilla y León (referencia para la tabla de tarifas): <https://itvcitaprevia.es/precios-itv/castilla-y-leon/>

JavaFX y frontend

- Documentación oficial de JavaFX: <https://openjfx.io/openjfx-docs/>
- Scene Builder — descarga y documentación: <https://gluonhq.com/products/scene-builder/>
- JavaFX SDK — descarga: <https://gluonhq.com/products/javafx/>
- Documentación oficial de Visual Studio Code: <https://code.visualstudio.com/>
- Thymeleaf — librería para generación de plantillas HTML: <https://mvnrepository.com/artifact/org.thymeleaf/thymeleaf/3.1.5.RELEASE>

- OpenHTMLtoPDF Core — conversión de HTML a PDF:
<https://mvnrepository.com/artifact/com.openhtmltopdf/openhtmltopdf-core/1.0.10>
- OpenHTMLtoPDF PDFBox — conector PDFBox para escritura de PDF:
<https://mvnrepository.com/artifact/com.openhtmltopdf/openhtmltopdf-pdfbox/1.0.10>
- Attoparser — análisis de HTML para Thymeleaf:
<https://mvnrepository.com/artifact/org.attoparser/attoparser/2.0.7.RELEASE>
- Unescape — escapado de caracteres especiales en HTML:
<https://mvnrepository.com/artifact/org.unescape/unescape/1.1.6.RELEASE>

Datos de la DGT

- Página oficial de la DGT — Parque de vehículos mensual:
<https://www.dgt.es/menusecundario/dgt-en-cifras/matraba-listados/parque-vehiculos-mensual.html>

ANEXOS

Anexo I – Desarrollo de las tareas realizadas

Tareas	Reparto
Arquitectura general de la aplicación	Ambos
<i>PythonAnywhere</i>	Juan Francisco
Base de datos	David
Interfaz <i>JavaFX</i>	David
Informe	Juan Francisco

Anexo II – Uso de la Inteligencia Artificial en el desarrollo del proyecto

1. Herramientas de inteligencia artificial utilizadas

A lo largo del desarrollo del proyecto hemos hecho uso de tres herramientas de inteligencia artificial con perfiles de uso distintos:

- Claude (*Anthropic*) ha sido la herramienta principal durante el desarrollo del *backend*. La hemos utilizado para resolver dudas técnicas sobre Python, depurar errores, contrastar decisiones de diseño y profundizar en conceptos que requerían un desarrollo matemático o teórico más elaborado, como el sistema de generación de matrículas mediante hash.
- GitHub Copilot ha actuado como apoyo puntual, especialmente durante el trabajo con el algoritmo de hash, donde lo consultamos para contrastar probabilidades de colisión, alternativas matemáticas y estadísticas relacionadas con la paradoja del cumpleaños.
- Gemini (*Google*) lo utilizamos principalmente en la fase de *frontend*, para orientarnos en el uso de JavaFX y Scene Builder, resolver problemas de conexión con la base de datos y apoyar el desarrollo de diversas funcionalidades de la interfaz.

2. Momentos del proyecto en los que se han utilizado

Es importante aclarar que la arquitectura de la aplicación, el diseño de la base de datos, el diseño de la interfaz y la estructura general de los scripts son de elaboración propia. La IA se ha utilizado como herramienta de apoyo, no como sustituta del trabajo de desarrollo.

En el *backend*, Claude se empleó principalmente para resolver errores concretos que surgían durante la ejecución de los scripts, como el error `max_allowed_packet` de MySQL al procesar las bajas o los problemas de codificación de caracteres en los CSVs; para contrastar decisiones técnicas ya tomadas, como la elección de `iterrows()` frente a `apply()` en pandas o la forma más eficiente de extraer componentes de un hash MD5; y para profundizar en conceptos que íbamos descubriendo durante el desarrollo, como el funcionamiento de las transacciones en

MySQL, el uso de `defaultdict`³ o el comportamiento del bloque `if __name__ == "__main__":`⁴.

Para el cálculo matemático relacionado con las colisiones de matrículas consultamos las tres herramientas obteniendo en todos los casos resultados coincidentes, lo que nos dio confianza en la solidez del modelo matemático aplicado. Todas aplicaron correctamente la fórmula de la paradoja del cumpleaños y llegaron a estimaciones similares, lo que nos permitió validar que el modelo era sólido independientemente de la herramienta consultada.

En el *frontend*, Gemini nos orientó en distintos momentos del desarrollo: desde las opciones de diseño disponibles en Scene Builder y la conexión de esta herramienta con Visual Studio Code, hasta la resolución del problema de conexión con la base de datos de *PythonAnywhere* a través del puerto 3306. También nos asistió en la generación de algunas clases POJO, en el desarrollo del CSS del informe PDF, en la identificación de las librerías necesarias para generar el PDF, en la implementación del guardado del PDF en la carpeta de descargas del dispositivo donde se ejecuta la aplicación, y en el uso de `showAndWait` para reflejar correctamente los cambios al cerrar una ventana.

3. Reporte de Prompts

A continuación, se recogen algunos de los momentos más representativos del uso de IA en el proyecto, seleccionados por su relevancia técnica o por el aprendizaje que generaron:

Prompt 1: Opciones de hash y colisiones esperadas (Copilot)

Durante el diseño del sistema de generación de matrículas consultamos a Copilot sobre las distintas opciones de implementación del hash y sobre cuántas colisiones cabría esperar con 82.000 registros. La respuesta nos aclaró un punto fundamental: el problema de las colisiones no está en el algoritmo de hash elegido (MD5, SHA-256 u otro), sino en el tamaño del espacio de salida. Con el formato NNNNLLL y un espacio de 156 millones de combinaciones posibles, la fórmula de la paradoja del cumpleaños predice aproximadamente 21 colisiones esperadas para 82.000 registros, y ese resultado es el mismo independientemente de si se usa MD5 o SHA-256 (evidentemente, estos cálculos se hicieron antes de dividir las matrículas entre las anteriores al 2000 y las posteriores). Esta consulta fue determinante para entender que el cuello de botella era el formato de la matrícula, no el algoritmo.

³ `defaultdict` es una subclase del diccionario de Python que asigna automáticamente un valor por defecto a cualquier clave que no existe, evitando el error que produciría un diccionario normal al acceder a una clave ausente. En `procesarCSV.py` lo utilizamos de dos formas: para el mapa de instancias, donde cada hash nuevo arranca automáticamente en 0 sin necesidad de comprobarlo antes de incrementar, y para el registro de matrículas usadas, donde cada matrícula nueva arranca también en 0 para controlar cuántas veces se ha generado y gestionar así los sufijos de colisión.

⁴ El bloque `if __name__ == "__main__":` controla si el código de un script se ejecuta o no dependiendo de cómo se haya invocado. Cuando Python ejecuta un fichero directamente, le asigna internamente el nombre especial `__main__`; cuando ese mismo fichero es importado desde otro script, recibe su propio nombre como módulo. En el proyecto, este bloque permite que el coordinador importe las funciones de los cuatro scripts del pipeline sin que estos se ejecuten automáticamente al ser importados, evitando así que una simple importación desencadene, por ejemplo, una descarga de datos o una inserción en la base de datos.

vale, y con tus tres mejores, cuales son las posibilidades de colision en 82 mil registros?

Buena, esta es la pregunta clave: **si afinamos al máximo el diseño, cuánto ganamos de verdad?**

Te resumo primero la idea importante, y luego te doy el número:

- Tus 82.000 registros **no compiten en el espacio MD5/SHA-256**, compiten en el espacio de **matrículas posibles**.
- Mientras sigas usando un formato `NNNNLLL` (156 millones de combinaciones), **da igual** que vengas de MD5, SHA-256, sal, bytes distintos... el límite lo marca **el tamaño del espacio de salida**, no el hash.

Números con 82.000 registros y matrículas modernas `NNNNLLL`

Espacio de salida:

$$m = 10^4 \cdot 25^3 = 156\,250\,000$$

Registros:

$$n = 82\,000$$

Número esperado de colisiones (da igual MD5, SHA-256, etc. mientras sean "aleatorios"):

$$E[\text{colisiones}] \approx \frac{n(n-1)}{2m} \approx 21$$

- Con el diseño actual → ~21 colisiones esperadas.
- Con SHA-256 + mejor extracción + sal → **siguen siendo ~21**.

Prompt 2: Cálculo matemático definitivo de colisiones (Claude)

Una vez teníamos el sistema definitivo con aproximadamente 1.875.000 registros y los dos alfabetos utilizados, pedimos a Claude que hiciera los cálculos matemáticos de colisiones esperadas distinguiendo entre matrículas modernas y antiguas. El espacio de combinaciones para las matrículas modernas con 21 letras es de 92.610.000 (10.000×21^3), y para las antiguas de 50.000.000 (8 provincias $\times 10.000 \times 25^2$). Asumiendo un 80% de vehículos modernos (1.500.000 registros) y un 20% de antiguos (375.000), la predicción teórica apuntaba a unas 12.150 colisiones en modernas (0,81%) y unas 1.406 en antiguas (0,37%), lo que daba un total estimado de 13.556 colisiones sobre el total de registros. El resultado real obtenido —14.069 colisiones, un 0,75%— se ajustó con precisión a la predicción teórica.

Y ahora, podrías hacer los cálculos matemáticos con las nuevas actualizaciones? No quiero los números exactos de registros, sino aproximaciones, porcentajes.. etc

Calculó colisiones de matrículas aplicando paradoja cumpleaños >

Aquí van los cálculos:

ESPACIO DE COMBINACIONES

Matrículas modernas (desde 2000) — formato **NNNNLLL** :

$10.000 \times 21 \times 21 \times 21 = 10.000 \times 9.261 = 92.610.000$ combinaciones posibles

Matrículas antiguas (antes de 2000) — formato **LLNNNNLL** :

Las dos primeras letras son fijas por provincia, por lo que:

$10.000 \times 25 \times 25 = 10.000 \times 625 = 6.250.000$ combinaciones por provincia

Con 8 provincias: $8 \times 6.250.000 = 50.000.000$ combinaciones posibles

COLISIONES ESPERADAS — PARADOJA DEL CUMPLEAÑOS

La fórmula es: **$\text{colisiones} \approx n^2 / (2 \times N)$**

Asumiendo aproximadamente un 80% de vehículos modernos y un 20% de antiguos sobre 1.875.000 registros:

Modernas (~1.500.000 registros):

$1.500.000^2 / (2 \times 92.610.000) \approx 12.150$ ↓ iones → ~0,81% del total

Prompt 3: Error *max_allowed_packet* en MySQL (Claude)

Al ejecutar el script de inserción en base de datos con el volumen completo de registros habiendo ya registros previamente, obtuvimos un error de MySQL al intentar marcar las bajas con un único UPDATE que incluía casi dos millones de matrículas en la cláusula IN. La consulta resultante superaba el límite de tamaño de paquete que permite MySQL por defecto (*max_allowed_packet*). Claude identificó la causa del error y propuso la solución de procesar las bajas en lotes de 1.000 registros, dividiendo la consulta masiva en múltiples consultas más pequeñas. La implementamos y resolvió el problema completamente.

He ejecutado manualmente mis script coordinador y ha habido este error en base de datos:

```
=====
=== PASO: 4. Inserción en base de
datos=====
===== INSERCIÓN EN BASE DE DATOS - DGT CyL ===[INFO] 8 CSV(s)
encontrado(s): · datos_BURGOS.csv · datos_LEÓN.csv · datos_PALENCIA.csv
· datos_SALAMANCA.csv · datos_SEGOVIA.csv · datos_SORIA.csv ·
datos_VALLADOLID.csv · datos_ZAMORA.csv[INFO] Conexión establecida
Mostrar más
```

Identificó causa de error y propuso solución por lotes >

El error es claro y está en `marcar_bajas()`. El problema es que estamos construyendo un único `UPDATE ... WHERE MATRICULA IN (...)` con todas las bajas de golpe, y la consulta resultante supera el límite de tamaño de paquete que permite MySQL (`max_allowed_packet`).

Con casi 2 millones de registros en base de datos y datos nuevos entrando, el conjunto de bajas puede ser enorme y la consulta pesa demasiado.

Prompt 4: Script de comprobación de matrículas duplicadas (Claude)

Durante la fase de desarrollo del sistema de hash, antes de tener la lógica de control de colisiones completamente implementada, necesitábamos una forma rápida de verificar si las matrículas que estábamos generando contenían duplicados. Pedimos a Claude un script de usar y tirar que levara el fichero TXT intermedio con todas las matrículas generadas y comprobara si había repeticiones. El script agrupaba las matrículas con las líneas donde aparecían y mostraba tanto el total de matrículas únicas como el listado de las repetidas con sus números de línea, lo que nos permitía ir directamente a los registros problemáticos para analizarlos. Esta herramienta fue clave para detectar y corregir los primeros fallos en el sistema de generación.

Bien bien. Ahora vamos a hacer una comprobación. Puedes hacerme un código de comprobación, de usar y tirar, para comprobar que en el TXT que hemos generado no hay ninguna matrícula repetida?

Ideó script para validar duplicados de matrículas >

Simple y directo. Lee el fichero, agrupa las matrículas con las líneas donde aparecen y al final:

- Si no hay duplicados → `✓ Sin duplicados. 82273 matrículas únicas.`
- Si los hay → lista cada matrícula repetida con sus números de línea para que puedas ir a mirarlás directamente.



Comprobar matrículas
PY

 Abrir en Code.exe

Prompt 5: Conexión con PythonAnywhere a través del puerto 3306 (Gemini)

Durante el desarrollo del *frontend* encontramos un error al intentar conectar la aplicación JavaFX con la base de datos de *PythonAnywhere*: el puerto 3306, que es el estándar de MySQL y el que habíamos utilizado sin problemas en proyectos anteriores, no permitía conexiones externas directas en *PythonAnywhere* por razones de seguridad de la plataforma. Consultamos a Gemini sobre el error y nos orientó hacia la documentación oficial de *PythonAnywhere*, que explica el mecanismo de conexión correcto para aplicaciones externas. Siguiendo esas instrucciones resolvimos el problema y establecimos la conexión de forma estable.

✦ ¡Tienes toda la razón en el cabreo! Es un muro técnico muy frustrante porque, aunque tu código sea perfecto, el problema es de **infraestructura externa**.

Aquí tienes la simulación de esa consulta técnica para tu documento del TFG, centrada específicamente en el bloqueo de **PythonAnywhere**:

Consulta: Error Crítico de Conexión (Puerto 3306 - PythonAnywhere)

Tu duda: "¡Ostras! No hay manera. He subido la base de datos a PythonAnywhere pero cuando intento conectar desde mi Java en VS Code me dice que el puerto 3306 está bloqueado o que la conexión ha sido rechazada. El año pasado hice un proyecto y me funcionaba, ¿qué está pasando?"

Mi respuesta (La solución técnica y corrección de conceptos):

"El problema no es tu código, sino una restricción de seguridad de **PythonAnywhere**.

1. **El Bloqueo:** PythonAnywhere **no permite conexiones externas entrantes** a sus bases de datos MySQL en las cuentas gratuitas (y en la mayoría de las básicas). Solo puedes conectar a esa base de datos si tu programa también está alojado dentro de sus servidores (por ejemplo, una web en Python).
2. **¿Por qué antes te funcionaba?:** Probablemente el año pasado trabajaste con un servidor **local** (como XAMPP, WAMP o una instalación directa de MySQL en tu PC). En local, el puerto 3306 está abierto para ti, pero en un servidor en la nube como PythonAnywhere, ellos cierran ese puerto para evitar ataques de fuerza bruta.

Solución aplicada para el TFG:

- **Cambio a Entorno Local:** Decidimos migrar la base de datos a un servidor local (MySQL Installer/Workbench) para el desarrollo del TFG. Esto garantiza que el puerto 3306 esté siempre disponible y que la latencia sea cero.
- **Refactorización de la clase `Conexion.java`:** Configuramos la URL como

Prompt 6: Guardado del PDF en la carpeta de descargas del dispositivo (Gemini)

Al implementar la generación del informe PDF necesitábamos que el fichero se guardara automáticamente en la carpeta de descargas del dispositivo donde se estuviera ejecutando la aplicación, independientemente del sistema operativo. Consultamos a Gemini cómo obtener esa ruta de forma dinámica en Java y nos proporcionó la solución para que la aplicación detecte la carpeta de descargas del usuario en tiempo de ejecución, garantizando así que el PDF siempre se guarda en el lugar correcto sin necesidad de configuración manual.

2. Que se "descargue" en el ordenador

Actualmente se guarda en la carpeta del proyecto porque le estarás pasando una ruta relativa.

Para que el usuario lo sienta como una "descarga", lo ideal es mandarlo a la carpeta de

Descargas del sistema.

En la variable `rutaDestino` que te puse arriba, `System.getProperty("user.home")` buscará automáticamente la carpeta del usuario (`C:/Users/Nombre/Downloads`).

4. Aportación de la IA al proyecto

El uso de la IA durante el proyecto ha supuesto un complemento al aprendizaje adquirido en clase, permitiéndonos profundizar en áreas que el ciclo formativo no cubre con el mismo nivel de detalle.

El primer aprendizaje relevante ha sido la potencia del ecosistema de librerías de Python. Conocíamos el lenguaje a nivel básico, pero el trabajo con `pandas`, `hashlib`, `BeautifulSoup`, `MySQL-connector` o `zipfile` nos ha mostrado cómo Python permite resolver problemas complejos de procesamiento de datos con un código relativamente compacto y legible, algo que habría resultado considerablemente más laborioso en Java.

El segundo aprendizaje ha sido el funcionamiento interno de MySQL ante volúmenes de datos elevados. Conocíamos el uso básico de consultas y teníamos noción del comportamiento de las transacciones con `commit` y `rollback`, pero nunca habíamos trabajado con tablas de casi dos millones de registros. Problemas como el `max_allowed_packet` al lanzar un `UPDATE` masivo, o la restricción de claves foráneas al intentar hacer un `TRUNCATE`, son situaciones que solo emergen a esa escala y que hemos tenido que resolver en tiempo real durante el desarrollo.

El último, y probablemente el más significativo, ha sido la aplicación del hash criptográfico a un problema real. El concepto de hash y el algoritmo MD5 no eran nuevos para nosotros⁵, pero nunca los habíamos utilizado de esta forma ni a este nivel de complejidad. Partir de una necesidad práctica concreta (identificar de forma estable vehículos cuyos datos son completamente anónimos) y llegar a un sistema de doble hash con control de instancias, gestión de colisiones y modelado matemático de probabilidades ha supuesto dar un salto cualitativo

⁵ Ya los habíamos trabajado en la última práctica de la asignatura de PSP, por lo que desde el primer momento decidimos usar MD5 para generar un código único con el que trabajar.

respecto al uso que habíamos hecho hasta ahora de esta herramienta (a lo que hay que unir las indicaciones de nuestras tutoras).